Demo Notes API: API Reference and Developer Guide

Document type: API reference

Audience: Developers learning to call a REST API

Version: 1.0

Status: Fictional portfolio demo

Important: *Notes API is a fictional example created for this portfolio project. The endpoints below are illustrative and are not connected to a live service. Do not send real credentials or sensitive data.*

1. Overview

The Demo Notes API illustrates how a client can create, retrieve, update, and delete simple text notes using HTTP and JSON.

Base URL

```
https://api.example.test/v1
```

This reserved example hostname is a placeholder. Replace it with the base URL of a real test server before executing requests.

2. Concepts

- **Resource:** An item managed by the API. In this example, a note.
- **Endpoint:** A URL and HTTP method used to perform an operation.
- **Request body:** JSON data sent by the client.
- **Response body:** JSON data returned by the API.
- **Status code:** A number indicating the result of a request.

3. Data model

A note has the following fields:

Field	Type	Required	Description
`id`	string	Response only	Unique identifier assigned to the note.
`title`	string	Yes when creating	Note title. Must contain at least one non-whitespace character.
`body`	string	Yes when creating	Note content. An empty string is allowed in this demo.

Field	Type	Required	Description
`createdAt`	string	Response only	Creation timestamp in ISO 8601 format.
`updatedAt`	string	Response only	Most recent update timestamp in ISO 8601 format.

Example note

```
{
  "id": "note_123",
  "title": "Meeting preparation",
  "body": "List the questions for the project review.",
  "createdAt": "2026-10-28T10:00:00Z",
  "updatedAt": "2026-10-28T10:00:00Z"
}
```

4. Endpoints at a glance

Method	Endpoint	Purpose	Typical success response
`GET`	`/notes`	List notes	`200 OK`
`GET`	`/notes/{id}`	Retrieve one note	`200 OK`
`POST`	`/notes`	Create a note	`201 Created`
`PATCH`	`/notes/{id}`	Update supplied fields	`200 OK`
`DELETE`	`/notes/{id}`	Delete a note	`204 No Content`

5. List notes

Retrieves the notes available to the caller.

Request

```
GET /v1/notes HTTP/1.1
Host: api.example.test
Accept: application/json
```

Example response — `200 OK`

```
{
  "data": [
    {
      "id": "note_123",
      "title": "Meeting preparation",
      "body": "List the questions for the project review.",
      "createdAt": "2026-10-28T10:00:00Z",
      "updatedAt": "2026-10-28T10:00:00Z"
    }
  ],
  "count": 1
}
```

The example response uses a `data` array and a `count` field. A real implementation must define and consistently follow its response schema.

6. Retrieve one note

Returns the note that matches the supplied ID.

Path parameter

Parameter	Type	Required	Description
`id`	string	Yes	Identifier of the note to retrieve.

Request

```
GET /v1/notes/note_123 HTTP/1.1
Host: api.example.test
Accept: application/json
```

Example response — `200 OK`

```
{
  "id": "note_123",
  "title": "Meeting preparation",
  "body": "List the questions for the project review.",
  "createdAt": "2026-10-28T10:00:00Z",
  "updatedAt": "2026-10-28T10:00:00Z"
}
```

7. Create a note

Creates a note from a JSON request body.

Request

```
POST /v1/notes HTTP/1.1
Host: api.example.test
Content-Type: application/json
Accept: application/json

{
  "title": "Meeting preparation",
  "body": "List the questions for the project review."
}
```

Example response — `201 Created`

```
{
  "id": "note_123",
  "title": "Meeting preparation",
  "body": "List the questions for the project review.",
  "createdAt": "2026-10-28T10:00:00Z",
  "updatedAt": "2026-10-28T10:00:00Z"
}
```

Validation rules in this demo

- Include `title` and `body`.
- Send both fields as strings.
- Do not send `id`, `createdAt`, or `updatedAt` when creating a note; the server assigns these fields.

8. Update a note

Updates only the fields supplied in the request. Fields omitted from the request remain unchanged.

Request

```
PATCH /v1/notes/note_123 HTTP/1.1
Host: api.example.test
Content-Type: application/json
Accept: application/json

{
  "title": "Project review preparation"
}
```

Example response — `200 OK`

```
{
  "id": "note_123",
  "title": "Project review preparation",
  "body": "List the questions for the project review.",
  "createdAt": "2026-10-28T10:00:00Z",
  "updatedAt": "2026-10-28T10:15:00Z"
}
```

A real implementation should define whether an empty patch is accepted and how invalid field values are handled.

9. Delete a note

Deletes the note identified by the path parameter.

Request

```
DELETE /v1/notes/note_123 HTTP/1.1
Host: api.example.test
```

Example response — `204 No Content`

The server returns no response body when deletion succeeds.

Caution: *Deletion is destructive. Confirm that the ID is correct before sending a DELETE request.*

10. Error responses

The following status codes are recommended for this demo contract. A real API must implement and document its actual behaviour.

Status	Meaning	Example cause	Client action
`400 Bad Request`	The request is invalid.	Required field is missing or has the wrong type.	Check the JSON body and field types.
`404 Not Found`	The requested resource does not exist.	The note ID is unknown.	Check the URL and note ID.
`415 Unsupported Media Type`	The server does not accept the request format.	`Content-Type` is missing or not JSON for a JSON body.	Set `Content-Type: application/json`.
`500 Internal Server Error`	The server encountered an unexpected error.	An unhandled server-side failure occurred.	Retry only when appropriate; report the request details without exposing secrets.

Illustrative error body

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "The title field is required."
  }
}
```

11. Authentication and security

This demo does not define a real authentication system. Do not add a fake API key or send credentials to the placeholder hostname.

For a production API, document the actual authentication scheme, required headers, token expiry behaviour, permissions, rate limits, and safe handling of secrets. Never place real credentials in source control.

12. Workflow example

```
Create note
    ↓
Read response and save the returned id
    ↓
Retrieve note by id
    ↓
Patch a field if needed
    ↓
Delete the note only when no longer needed
```

13. Success criteria

A developer can use this reference successfully when they can identify the correct method and endpoint, construct the expected request, interpret the documented response, and choose an appropriate action for a documented error.

14. Scope and assumptions

This is a documentation-only sample for a fictional REST API. It is not a working API, and its example responses are proposed contract examples rather than observed server output. A complete production reference should be validated against a running implementation and include a tested OpenAPI specification.

End of API reference